BITS PILANI · WORK INTEGRATED LEARNING PROGRAMME

Heart Disease Prediction MLOps Final Report

An end-to-end, reproducible machine-learning workflow covering data analysis, model development, experiment tracking, automated validation, API serving, containerization, orchestration and monitoring.

AIMLCZG523 · Machine Learning Operations

M.Tech Artificial Intelligence & Machine Learning

Binary classification · UCI Cleveland dataset

Repository: github.com/dwai86/mlops-heart-disease · Report edition: October 2026

Submitted by: Dwaipayan De

✓ **Verified project state · 14 October 2026**

The saved EDA notebook contains executed outputs; the BITS lab's Python 3.12 run passed all 7 tests and completed model training; the refreshed artifact returned successful `/health`, `/predict` and `/metrics` responses in the Python 3.11 Docker container. GitHub Actions run #9 completed successfully, including lint, unit tests and training on Python 3.11. The repository includes Docker packaging and Kubernetes deployment configuration; the BITS lab validation documented here is Docker-based.

BITS lab evidence. The test, training, Docker build, API validation and container log evidence is available in `screenshots/bits lab evidence/`.

DATASET	INPUT FEATURES	MODELS COMPARED	LOCAL TESTS
303	13	2	7 / 7
Cleveland subset records	Numeric and categorical	Logistic Regression · Random Forest	Passed; deprecation warnings only

Executive Summary

PROJECT OVERVIEW

This project implements an end-to-end MLOps workflow for binary classification on the UCI Cleveland Heart Disease dataset. Its objective is to demonstrate repeatability and operational practices around data preparation, exploratory data analysis (EDA), preprocessing, model comparison, experiment tracking, packaging, automated checks, API serving and runtime observability.

The training code compares Logistic Regression and Random Forest. A stratified 80/20 train/test split uses a fixed random state, and five-fold StratifiedKFold cross-validation supports hyperparameter search. The selection criterion is mean cross-validation ROC-AUC. A scikit-learn `ColumnTransformer` now treats numeric and categorical inputs separately: numeric columns are median-imputed and scaled, while categorical columns are imputed with the most-frequent value and one-hot encoded. The preprocessing steps remain inside the model pipeline to reduce validation leakage.

MLflow records model parameters, cross-validation and held-out test metrics, generated plots and serialized model artifacts. The selected pipeline is saved using joblib and served through FastAPI via `/predict`, with `/health` and Prometheus-compatible `/metrics` endpoints. GitHub Actions runs linting, tests and model training; Docker and local Kubernetes manifests support deployment validation.

> **Scope and safety.** This is an academic MLOps demonstration using a small historical dataset. The output must not be used for clinical diagnosis, triage or treatment decisions.

01 Problem Definition

1.1 Objective

Design a reproducible machine-learning service that accepts 13 clinical attributes and classifies a record into a binary target: no disease (0) or disease present (1). The work emphasizes the operational lifecycle rather than only model fitting: dependency management, repeatable training, tracking, testing, serving, containerization and monitoring.

1.2 Problem formulation

ITEM	DEFINITION
Learning task	Supervised binary classification
Dataset	UCI Heart Disease dataset, Cleveland subset
Records	303 records in the processed Cleveland file
Predictors	13 input features
Original diagnosis field	Values 0–4
Binary target	0 remains 0; any original diagnosis value greater than 0 maps to 1
Primary model-selection metric	Mean five-fold cross-validation ROC-AUC
Final evaluation	Held-out stratified test split

1.3 End-to-end lifecycle

1. **Acquire & inspect** — Source dataset, schema, missing values
2. **Explore & prepare** — EDA, target mapping, feature types
3. **Train & compare** — CV, GridSearchCV, MLflow
4. **Package & serve** — joblib pipeline, FastAPI
5. **Validate changes** — Ruff, pytest, CI training
6. **Containerize** — Docker image and API runtime
7. **Orchestrate** — Kubernetes Deployment and Service
8. **Observe** — Request logs and Prometheus metrics

02 Data Acquisition, Cleaning & EDA

2.1 Dataset acquisition

The project uses the processed Cleveland file located at `data/raw/heart+disease/processed.cleveland.data`. The repository includes a data acquisition/validation script, and the training loader checks that the input has the expected 14 columns. The raw diagnosis field is converted to a binary target before fitting.

2.2 Data cleaning and target transformation

Missing fields represented by `?` are read as missing values by pandas. Missing predictors are imputed within the model pipeline rather than by fitting preprocessing globally before the train/test split. The original target is transformed so that value 0 becomes class 0 and all values greater than 0 become class 1.

2.3 EDA completed in the notebook

The saved notebook `notebooks/eda.ipynb` contains executed outputs for the dataset preview, class proportions, missing-value counts, numeric histograms and correlation heatmap. The processed file has **303 records** and **13 predictors**; the binarized target contains **164 class-0 records (54.1%)** and **139 class-1 records (45.9%)**. Six predictor values are missing: `ca` has 4 and `thal` has 2; no other feature is missing.

Numerical distribution

Histograms cover age, resting blood pressure (`trestbps`), cholesterol (`chol`), maximum heart rate (`thalach`) and ST depression (`oldpeak`).

Data quality and label balance

The notebook examines missingness and the transformed binary target distribution, supporting decisions about preprocessing and evaluation metrics.

Binary target balance

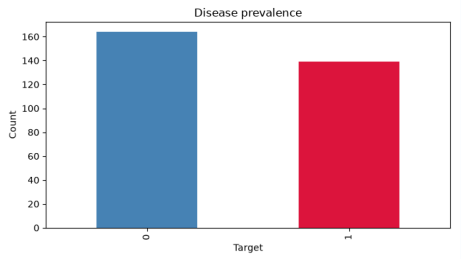


Figure 2. Target distribution for all 303 Cleveland records; class 1 represents any original target value greater than zero.

Feature correlation

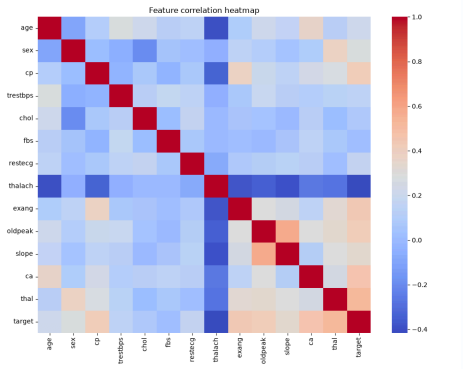
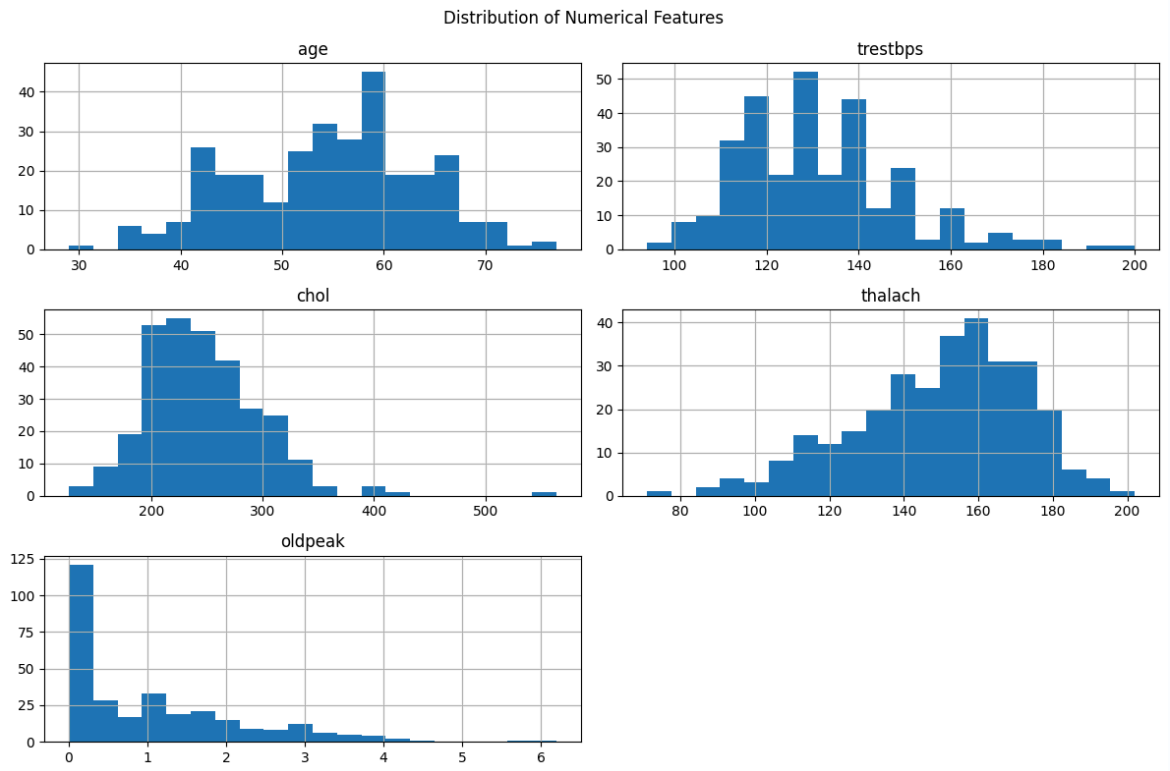


Figure 3. Correlation heatmap; pairwise correlation is descriptive and does not establish causality.

Numeric feature distributions



Supplemental EDA visualization. The five histograms show varied scales and distribution shapes; each plotted numeric feature has 303 non-missing observations.

Data-quality interpretation. The class proportions are reasonably close in this subset, while the six missing predictor values are handled inside the training pipeline. Numeric features span different scales, motivating standardization for Logistic Regression. EDA figures are descriptive and are not used as a substitute for held-out evaluation.

2.4 Why EDA matters

EDA helps identify incomplete records, understand variable scales, confirm the target transformation, and reveal possible imbalance. The findings inform the preprocessing pipeline and the choice to report precision, recall, F1 and ROC-AUC alongside accuracy.

03 Feature Engineering & Preprocessing

3.1 Feature groups

TYPE	FEATURES	TRANSFORMATION
Numerical	`age`, `trestbps`, `chol`, `thalach`, `oldpeak`	Median imputation, then StandardScaler
Categorical / coded categories	`sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `ca`, `thal`	Most-frequent imputation, then OneHotEncoder

3.2 Preprocessing architecture

A scikit-learn `ColumnTransformer` applies feature-type-specific transformations. The numerical pipeline uses `SimpleImputer(strategy="median")` and `StandardScaler`. The categorical pipeline uses `SimpleImputer(strategy="most_frequent")` and `OneHotEncoder(handle_unknown="ignore")`. Both are embedded in a single model `Pipeline`.

```
Raw feature DataFrame
        |
        ▼
ColumnTransformer
  ├── Numeric: median imputation → StandardScaler
  └── Categorical: most-frequent imputation → OneHotEncoder
        |
        ▼
Classifier (Logistic Regression or Random Forest)
        |
        ▼
Fitted pipeline serialized with joblib
```

3.3 Leakage prevention and reproducibility

The split occurs before model fitting. Since imputation, scaling and encoding are steps of the estimator pipeline passed to GridSearchCV, each cross-validation fold fits preprocessing only on its own training fold. The held-out test data is reserved for final evaluation. A fixed random state is used for the train/test split and the shuffled five-fold StratifiedKFold setup.

> **Implementation alignment:** This section reflects the updated source code in `src/heart_disease/data_pipeline.py`, including the numeric/categorical split and one-hot encoding.

04 Model Development & Evaluation

4.1 Candidate models

Logistic Regression provides a linear baseline, while Random Forest offers a nonlinear ensemble model. Both are trained and tuned using the same split strategy and preprocessing approach.

4.2 Hyperparameter search

MODEL	PARAMETER	SEARCH VALUES
Logistic Regression	`C`	0.1, 1.0, 10.0
Logistic Regression	`solver`	`liblinear`, `lbfgs`
Random Forest	`n_estimators`	100, 200, 300
Random Forest	`max_depth`	`None`, 5, 10
Random Forest	`min_samples_leaf`	1, 2, 4

GridSearchCV evaluates the candidate combinations with five-fold StratifiedKFold. The scoring dictionary includes accuracy, precision, recall, F1 and ROC-AUC, and the search refits the candidate with the highest ROC-AUC. Classifiers use a fixed random state; class weighting is set to `balanced`.

4.3 Model selection and metrics

Each model family produces its best cross-validation candidate. The overall pipeline is selected by comparing mean five-fold CV ROC-AUC: Logistic Regression scored **0.9031 ± 0.0166**, slightly above Random Forest at **0.9016 ± 0.0297**, so Logistic Regression is selected. Random Forest has higher held-out accuracy and recall in this split, but the test set is reserved for evaluation and is not used to choose or tune the model.

> **Best parameters in the refreshed artifacts.** Logistic Regression (selected): `C=0.1`, `solver=liblinear`. Random Forest: `n_estimators=200`, `max_depth=10`, `min_samples_leaf=2`. Both use `class_weight=balanced` and `random_state=42`. These values were read from the current saved model pipelines.

Latest model results · editable

METRIC	LOGISTIC REGRESSION	RANDOM FOREST
CV accuracy (mean ± SD)	**0.8304 ± 0.0168**	**0.8055 ± 0.0296**
CV precision (mean ± SD)	**0.8232 ± 0.0435**	**0.7921 ± 0.0528**
CV recall (mean ± SD)	**0.8099 ± 0.0680**	**0.7917 ± 0.0946**
CV F1-score (mean ± SD)	**0.8132 ± 0.0230**	**0.7865 ± 0.0408**
CV ROC-AUC (mean ± SD)	**0.9031 ± 0.0166**	**0.9016 ± 0.0297**
Test accuracy	**0.8852**	**0.9016**
Test precision	**0.8387**	**0.8438**
Test recall	**0.9286**	**0.9643**
Test F1-score	**0.8814**	**0.9000**
Test ROC-AUC	**0.9665**	**0.9535**

20 of 20 metric cells filled.

4.4 Metric interpretation

Accuracy and precision

Accuracy measures the fraction of correct predictions; precision measures the fraction of predicted positives that are truly positive.

Recall and F1

Recall measures how many actual positive records are identified. F1 balances precision and recall through their harmonic mean.

ROC-AUC

ROC-AUC measures ranking/discrimination across classification thresholds and is the primary CV selection criterion in this implementation.

Interpretation limit

These metrics characterize performance on the specified dataset and held-out split; any clinical application would require separate validation on appropriate patient populations.

05 Experiment Tracking & Packaging

5.1 MLflow experiment design

The training code configures the `heart-disease-prediction` experiment. Separate runs track Logistic Regression and Random Forest searches, followed by a final best-model run. Logged information includes chosen parameters, CV means and standard deviations for accuracy, precision, recall, F1 and ROC-AUC, held-out test metrics, confusion-matrix/probability plots and serialized model artifacts.

5.2 Saved artifacts and selected model

The latest local MLflow run selected **Logistic Regression** with `model__C=0.1` and `model__solver=liblinear`, based on its higher mean five-fold CV ROC-AUC (0.9031 vs. 0.9016). The final fitted pipeline was saved to `artifacts/best_model.joblib`. The stratified 20% hold-out contains 61 records; the selected model's confusion matrix is TN=28, FP=5, FN=2, TP=26.

ARTIFACT	PURPOSE	LATEST LOCAL CHECK
`artifacts/best_model.joblib`	Selected fitted preprocessing + classifier pipeline	Present · 5,110 bytes
`artifacts/logistic_regression.joblib`	Best Logistic Regression pipeline	Present · 5,110 bytes
`artifacts/random_forest.joblib`	Best Random Forest pipeline	Present · 1,190,411 bytes
`artifacts/logistic_regression_metrics.png`	Confusion matrix and predicted-probability distribution	Present · 28,430 bytes
`artifacts/random_forest_metrics.png`	Confusion matrix and predicted-probability distribution	Present · 27,390 bytes

Logistic Regression · held-out results

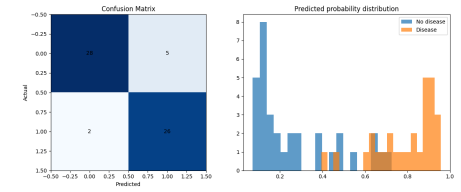


Figure 4. Test-set confusion matrix and predicted-probability distribution for the selected Logistic Regression pipeline.

Random Forest · held-out results

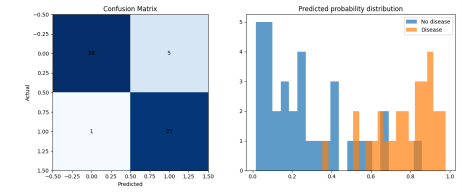


Figure 5. Corresponding Random Forest evaluation plot; its test scores do not determine model selection.

5.3 Reproducible storage behavior

The local MLflow store is configured at `file:./mlruns`. Machine-specific run metadata is not committed to Git; the repository ignores the local `mlruns/` directory. This avoids carrying Windows-specific artifact paths into another environment. A fresh environment should create its own tracking files by rerunning training.

> **Portability note.** Keep MLflow run output, exported metric values and the final report as deliberate deliverables/evidence. A local file store is useful for this assignment but is not a centralized experiment server.

06 FastAPI Model Serving

6.1 API endpoints

METHOD AND PATH	PURPOSE	LOCAL CHECK
`GET /health`	Health response for service availability	Passed
`POST /predict`	Accepts 13 feature fields as JSON and returns prediction, probability and risk label	Passed
`GET /metrics`	Exposes Prometheus-format request and latency metrics	Passed

6.2 Request fields and sample input

The request comprises `age`, `sex`, `cp`, `trestbps`, `chol`, `fbs`, `restecg`, `thalach`, `exang`, `oldpeak`, `slope`, `ca` and `thal`. Input field names are aligned with the training feature schema. The following sample is the payload used for the fresh local response check:

```
{
  "age": 63, "sex": 1, "cp": 1, "trestbps": 145, "chol": 233,
  "fbs": 1, "restecg": 2, "thalach": 150, "exang": 0,
  "oldpeak": 2.3, "slope": 3, "ca": 0, "thal": 6
}
```

6.3 Current local API response

```
{
  "prediction": 0,
  "probability": 0.4553644341348998,
  "risk_label": "Low risk"
}
```

Observed in a fresh local FastAPI TestClient request on 14 October 2026 using the current `best_model.joblib` and the sample payload above. This confirms the current local application path and inference response for the documented sample; results are presented for the academic project scope.

Interactive API response editor

Prediction (0/1)

Probability

Risk label

The values above are reflected in the response JSON and print view.

6.4 Operational note

The model artifact contains the trained preprocessing and classifier pipeline, allowing the API to receive raw feature values without duplicating preprocessing logic in the serving layer. The local request validates the API contract and inference path for the selected artifact.

07 CI/CD & Automated Testing

7.1 GitHub Actions workflow

The workflow uses GitHub Actions to automate code validation and model training. It checks out the repository, configures Python 3.11, installs dependencies, runs Ruff, executes pytest and runs the training entry point. A non-zero return from a validation or training command fails the job.

STAGE	COMMAND / IMPLEMENTATION	PURPOSE
Checkout	`actions/checkout@v4`	Fetch source for the workflow run
Runtime	Python 3.11 via `actions/setup-python@v5`	Standardize CI runtime
Dependencies	`pip install -r requirements.txt`	Install project dependencies
Lint	`ruff check .`	Check style and common code issues
Tests	`pytest -q`	Exercise data/model/API behavior
Training	`python scripts/train_model.py --data ... --artifact-dir artifacts`	Verify the training entry point executes

7.2 Local test and CI evidence

✓ **Validation passed in the BITS lab and GitHub Actions**

The BITS lab's Python 3.12 environment completed **7 tests passed, 14 dependency deprecation warnings**; the captured run took 16.69s. GitHub Actions run #9 completed successfully on 14 October 2026; its Ruff, pytest and model-training steps all passed on Python 3.11 (commit `1e481b9`).

BITS lab test evidence: `screenshots/bits lab evidence/pytest.txt` and `screenshots/bits lab evidence/pytest_success.png`.

7.3 Training outputs

The BITS lab training command completed and saved the refreshed model artifacts. Training evidence is available in `screenshots/bits lab evidence/training.txt` and `screenshots/bits lab evidence/model_training.png`. The GitHub Actions workflow also runs training after linting and unit tests.

08 Containerization & Deployment Validation

8.1 BITS lab Docker validation

The BITS lab built image `heart-disease-api:bits` using the Dockerfile's `python:3.11-slim` base. The container returned successful responses from `/health`, `/predict` and `/metrics`; the sample prediction was class 0 with probability 0.455364. Docker build, container start, health, prediction, metrics and container log evidence is stored in `screenshots/bits lab evidence/`. The Swagger API screenshot is `screenshots/bits lab evidence/swagger.png`; the Docker/API screenshot is `screenshots/bits lab evidence/fastapi_docker.png`.

8.2 Kubernetes configuration

The repository includes Kubernetes Deployment and Service manifests for cluster-based deployment.

ASPECT	DOCUMENTED SETUP	IMPLEMENTATION SUMMARY
Cluster	Local Kind cluster	Local Kubernetes environment for deployment validation
Workload	Deployment with two replicas	Two-replica Deployment configuration
Service	LoadBalancer	Kubernetes Service configured as part of the deployment setup
Endpoint access	`kubectl port-forward`	Used to access and validate API endpoints during local Kubernetes testing

09 Monitoring, Logging & Observability

9.1 Prometheus-compatible metrics

The API exposes `heart_disease_requests_total` and `heart_disease_request_latency_seconds` from `/metrics`. A fresh local TestClient request returned HTTP 200 and Prometheus-format output. The repository's `monitoring/prometheus.yml` targets `host.docker.internal:8000`; this is a local-Docker-oriented target

9.2 Request logging

API middleware records request timestamps, paths and response status codes in `logs/api.log`, providing a straightforward record for request tracing and local troubleshooting.

9.3 Monitoring scope

This implementation provides lightweight observability appropriate to the academic scope. More mature production operations could add Grafana dashboards, alerts, durable metrics storage, data-quality checks, drift monitoring, access controls and health probes.

> **Observability summary.** The API exposes request-count and latency metrics in Prometheus format through `/metrics`, and request middleware records API activity. Together, these provide a lightweight foundation for

runtime visibility and troubleshooting.

10 System Architecture

The architecture below summarizes the source-to-service lifecycle implemented in the repository. It distinguishes model development/experiment tracking from serving and runtime observability.

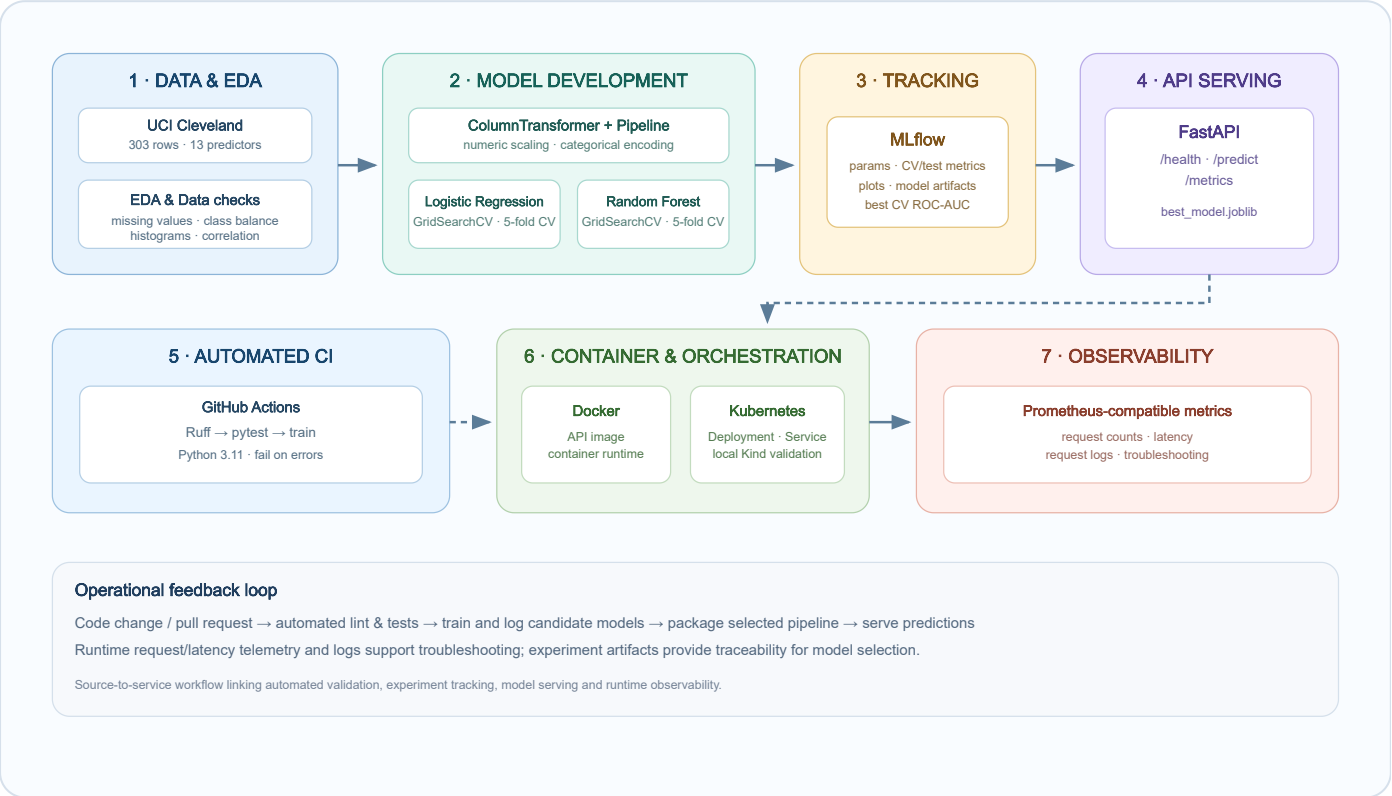


Figure 1. Heart Disease Prediction MLOps architecture, updated to reflect mixed numeric/categorical preprocessing. The standalone vector version is available as SVG.

11 Assignment Rubric Coverage

The matrix maps each requirement from the assignment instructions to the corresponding implementation and available project evidence, covering data preparation, modelling, experiment tracking, automated validation, serving, deployment and observability.

REQUIREMENT	MARKS	PROJECT IMPLEMENTATION / EVIDENCE	STATUS
Data acquisition & EDA	5	UCI Cleveland source file, download/validation script, executed `notebooks/eda.ipynb`, missing-value/class-balance analysis, histograms, heatmap	Implemented
Feature engineering & model development	8	Mixed numeric/categorical preprocessing, two classifiers, GridSearchCV, five-fold stratified CV, ROC-AUC selection, test metrics	Implemented
Experiment tracking	5	MLflow experiment, model parameters, CV/test metrics, plots and model artifacts	Implemented
Packaging & reproducibility	7	joblib pipelines, requirements, pyproject, fixed random seeds, local MLflow ignored by Git	Implemented
CI/CD & automated testing	8	GitHub Actions, Ruff, pytest and model training; 7 local tests passed	Core checks passed
Model containerization	5	Dockerfile and documented container/API validation	Implemented
Production deployment	7	Kubernetes Deployment/Service and local Kind validation using port-forwarding	Implemented
Monitoring & logging	3	API request logging, Prometheus-compatible endpoint and local metrics endpoint check	Verified locally
Documentation & reporting	2	README, final report, architecture diagram, deployment evidence and supporting media	Documented
Total	**50**	Rubric categories and weightings follow the assignment brief; the matrix maps each category to the implementation and supporting evidence.	

12 Setup & Reproducibility

12.1 Local setup

The repository README provides the primary setup sequence. In PowerShell, from the repository root:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip install -e .
python scripts/download_data.py
python scripts/train_model.py --data "data/raw/heart+disease/processed.cleveland.data" --artifact-dir
artifacts
pytest -q
```

If the dataset is already present, verify it is the intended 303-row Cleveland file before retraining. The project README is authoritative if a script's arguments or local paths differ.

12.2 Run the API

```
python -m uvicorn heart_disease.api:app --host 127.0.0.1 --port 8000
```

Verify `/health`, submit JSON to `/predict`, and inspect `/metrics`. Keep the API process in one terminal and execute requests from another. Use the project README and the selected lab environment to run the documented Docker and Kubernetes validation steps.

12.3 Local validation already performed

CHECK	OBSERVED OUTCOME
`pytest -q`	7 tests passed in the local environment
Local training entry point	Completed; MLflow logged both tuned models and selected the higher-CV-ROC-AUC pipeline
Artifact directory	Five expected model/plot files present with non-zero size
FastAPI `/health`	HTTP 200; `{"status":"ok"}`
FastAPI `/predict`	HTTP 200; prediction 0, class-1 probability 0.455364, label “Low risk” for the documented sample input
FastAPI `/metrics`	HTTP 200; Prometheus-format metrics returned
GitHub Actions run #9	Successful; Ruff, pytest and training steps passed on Python 3.11

13 Project Scope & Future Enhancements

The project is designed as a focused academic demonstration of an end-to-end MLOps lifecycle. Its current scope brings together the Cleveland dataset, a local MLflow file store, local Kind Kubernetes configuration and lightweight application observability.

- **Dataset extension:** The same preprocessing and evaluation workflow can be extended to additional datasets to study generalization across populations.
- **Clinical governance:** Any real-world clinical use would require independent clinical validation, privacy safeguards and relevant regulatory review. This model is intended for coursework only, not diagnosis or treatment.
- **Observability evolution:** Prometheus-compatible metrics and request logging provide a foundation for dashboards, alerts, drift monitoring and data-quality checks in a future iteration.
- **Deployment evolution:** The Kubernetes configuration supports local Kind validation; cloud-hosted deployment with ingress can be considered as a future extension.
- **Experiment tracking evolution:** The project uses a local MLflow file store for reproducible experiments. A shared tracking backend could be introduced if multiple environments or contributors need centralized access.
- **Operational maturity:** Future iterations could add authentication, resource limits, health/readiness probes, persistent log collection and model/version governance.

> **Conclusion.** The project brings together source-data validation, EDA, mixed-type preprocessing, model comparison and selection, MLflow tracking, a reusable inference pipeline, automated code checks, FastAPI serving, Docker

packaging, Kubernetes configuration and runtime metrics. Its modular design provides a practical foundation for further MLOps development.